

Guide progress

- ☐ Step 1 — Topic
- ☐ Step 2 — Data
- ☐ Step 3 — Characters
- ☐ Step 4 — Prompts
- ☐ Step 5 — Outputs

A step turns green only when all its checkboxes are marked.

Political Narratives Guide

Use this guide to understand what the concept of **Political Narrative** is and to organize the pipeline for your research. Step by step, it helps you to define your topic, choose sources, identify characters, design prompts, and generate results, while highlighting the key questions you should answer at each stage.

Steps

Intro	1 • Topic	2 • Data	3 • Characters	4 • Prompts
5 • Coding				

Political Narratives guide

A political narrative is identified by (i) its topic, (ii) its characters, and (iii) by having at least one character cast in a drama triangle role: hero, villain, or victim.

The definition and measurement of political narratives, therefore, reduce to specifying the topic and characters, and coding for each character whether it appears as neutral or cast as hero, villain, or victim. The purpose of a political narrative is influencing perceptions, beliefs, and preferences. Political narratives exert their influence by depicting characters - human or instrument - in one of the three archetypal roles — **hero**, **villain**, or **victim**. They are communicative devices that focus attention, encode roles and identities, and shape norms and behavior.

We can also depict this formally. To define and measure narratives, you need to choose a topic T and a universe of characters $K = H \cup I$, where H and I represent Human and Instrument characters. For any text unit (tweet, paragraph, article), let $K' \subseteq K$ be the set of characters that appear. A role-assignment function $r : K' \rightarrow \{\text{hero}, \text{villain}, \text{victim}, \text{neutral}\}$ maps each appearing character to either a drama-triangle role or neutrality. We call (T, K', r) a **political narrative** if and only if at least one character is cast as hero, villain, or victim. If all characters are neutral, the text is about the topic but does not constitute a political narrative.

How to use this guide

- Use the step selector above to move from **1** → **5**.
- Each step includes three cards:
 - **Guide**  — brief “How to” plus reflective **Ask yourself** items.
 - **Example**  — a concrete mini-case clarifying the step.
 - **Output**  — what you should have before moving on.
- Write your ideas in the **Notes** box at the end of each step and keep notes on your progress.

Guide progress

- ☐ Step 1 — Topic
- ☐ Step 2 — Data
- ☐ Step 3 — Characters
- ☐ Step 4 — Prompts
- ☐ Step 5 — Outputs

A step turns green only when all its checkboxes are marked.

Political Narratives Guide

Use this guide to understand what the concept of **Political Narrative** is and to organize the pipeline for your research. Step by step, it helps you to define your topic, choose sources, identify characters, design prompts, and generate results, while highlighting the key questions you should answer at each stage.

Steps

Intro

1 • Topic

2 • Data

3 • Characters

4 • Prompts

5 • Coding

Step 1 — Select and define the topic

A precise topic definition anchors character selection and downstream analysis.

Guide: define a clear topic 

How to approach

- A well-defined topic is a prerequisite for fruitful narrative analysis. The clearer the topic, the more straightforward the identification of relevant characters and the exploration of the research question. Topic choice should weigh the research question, data availability, and available resources, while balancing specificity vs. generalizability. Over-narrow topics risk too few characters or narratives; over-broad topics make it difficult to restrict analysis to a manageable set. Clearly document what aspects of the topic are included and excluded from the analysis.

Ask yourself

- ☐ Is the topic sufficiently specific to be analyzable, without being so narrow that it lacks variation?

- ☐ Does this topic surface enough distinct political narratives and public debate to analyze?
- ☐ Is it likely there are enough identifiable characters within those narratives? Characters can be individuals, parties, institutions, or groups appearing in the discourse.
- ☐ Is the research question compelling and relevant to the scientific community (and/or practitioners)?
- ☐ Which data sources are most informative for this topic, and do you have reliable access to them?
- ☐ If those sources are available, can you obtain the essential metadata (dates, outlets, geography, language) needed for analysis?
- ☐ Could any actors or communities be harmed by this analysis, and how would you mitigate that risk?

Example — Focusing on policy narratives within climate change



In *Gehring & Grigoletto (2025)* we analyze the **political economy of climate change**. From the literature and reading our potential sources, we realized there are two rather distinct political discussion – one about the scientific evidence behind the causes and consequences of climate change, one about policies to address climate change. Given our research interest is on the political reactions and discourse about political solutions, we define our topic more narrowly as **climate change policies**. We explicitly exclude debates on the scientific reality and predictability of climate change.

Output — What you should have before Step 2 ⚠️

- A precise topic for the analysis of political narratives. We suggest writing it up in full sentences, but also trying to define it via bullet points. Both a positive definition is useful (what should be included), as well as a negative distinction (e.g. it should NOT include certain related or overlapping topics). This topic defines where to look and what kind of material to collect, hence is the basis for Step 2 Sources and Extraction

Guide progress

- ☐ Step 1 — Topic
- ☐ Step 2 — Data
- ☐ Step 3 — Characters
- ☐ Step 4 — Prompts
- ☐ Step 5 — Outputs

A step turns green only when all its checkboxes are marked.

Political Narratives Guide

Use this guide to understand what the concept of **Political Narrative** is and to organize the pipeline for your research. Step by step, it helps you to define your topic, choose sources, identify characters, design prompts, and generate results, while highlighting the key questions you should answer at each stage.

Steps

Intro

1 • Topic

2 • Data

3 • Characters

4 • Prompts

5 • Coding

Step 2 — Identify the source and extract data

Choose sources (e.g., newspapers, social media, transcribed TV/radio/YouTube, surveys).

Guide: source selection & data extraction

How to approach

- After selecting the topic, the next step is gathering data. Common sources include digitized newspapers, social media, transcribed TV/radio/YouTube content, and open-ended survey responses.
- When selecting the data source, prioritize the media channels where narratives about your chosen topic are most prominent.
- Evaluate trade-offs between coverage, accessibility, and quality (e.g., digitization errors, platform bias, sampling limits).
- For data extraction, the source you choose will determine which methodologies can be applied: e.g. keyword-based queries, scraping, API pulls, or manual collection.
- Consider the level of metadata you can preserve (dates, outlets, authors, geography, language) since these details will later be

the basis of your analysis.

- What should be the unit of your analysis? A tweet, paragraph, newspaper snippet, or other? Ask yourself: Should I split the texts into smaller snippets, or can I work with the extracted texts as they are? For example, long texts (e.g., articles) can be segmented into paragraphs or 3-sentence chunks to fit LLM input limits and maintain focus.
- If text snippets are too short, you will capture mostly narrative fragments rather than full narratives with multiple character roles; if they are too long, the LLM may struggle to assign consistent roles—especially when a character is portrayed in different ways within a long text. From experience, units roughly the length of a tweet, paragraph, or short 2–3-sentence chunk work best, as they preserve the natural meaning of sentences and paragraphs while avoiding arbitrary cuts that break narrative structure.
- If very long texts cannot be avoided, one should consider specifying in the prompts at Step 4 how to cope with contradictory or changing roles over the course of the text.

Ask yourself

- ☐ Do the selected sources reflect the key (media) platforms where the political debate takes place?
- ☐ Is the time window covered by the source appropriate for the research question?
- ☐ Are the selected sources sufficiently diverse to avoid bias toward one outlet, ideology, or demographic?
- ☐ Do you have legal and technical access to these data (e.g., archives, APIs, scraping permission)? Ensure compliance with copyright and platform terms when extracting data.
- ☐ What extraction method is most reliable for your source? E.g., keyword queries, metadata filters, transcript parsing, etc.
- ☐ Is the extraction method able to produce snippets that are neither too short to lose context nor too long to become too complicated for the analysis? If needed, split the texts into smaller snippets.

- ☐ Have you applied basic preprocessing (e.g., removing duplicates, non-textual artifacts), and considered filtering or tagging by language if the texts are multilingual?

Example — Extracting data using Twitter Historical APIv2 💡

In *Gehring & Grigoletto (2025)* our focus is on narratives about climate change policies in the United States, collected from the social media platform Twitter. We specifically choose the U.S. due to the significant role Twitter plays in shaping and disseminating political narratives there. The data collection process involves querying the **Twitter historical APIv2** with a set of **keywords** adapted from *Oehl, Schaffer, and Bernauer (2017)*. In our main analysis, we define the tweet as our unit of observation, since its concise length aligns well with the requirements of the subsequent stages of the framework, including the GPT prediction process. We also extract newspaper articles and TV transcripts, which we decided to split into smaller snippets to make them compatible with the framework's next steps.

- **Tweets:** Tweets have been extracted through a keyword based search using the historical API of Twitter
- **Newspaper Articles:** Newspaper articles are downloaded from Factiva. We use the three most widely circulated newspapers in the US; The New York Times, The Wall Street Journal, and USA Today. Each article has been split into more manageable text snippets (three sentences paragraphs) for OpenAI classification.
- **TV Transcripts:** TV transcripts have been downloaded from the GDELT database. These TV transcripts from MSNBC and Fox News have been also split into smaller text snippets (three sentences paragraphs).

Output — What you should have before Step 3 ⚠️

- A dataset with the extracted text snippets and other metadata, if you need them for your analysis.
- Store your extracted data in a UTF-8 CSV with at least two columns: id and text, plus optional metadata (date, outlet, etc.).

	id	text	...
	1_1	If you listen...	...
	1_2	But is it po...	...
	1_3	In fact, on...	...
Personal notes for Step 2 (optional)			

Guide progress

- ☐ Step 1 — Topic
- ☐ Step 2 — Data
- ☐ Step 3 — Characters
- ☐ Step 4 — Prompts
- ☐ Step 5 — Outputs

A step turns green only when all its checkboxes are marked.

Political Narratives Guide

Use this guide to understand what the concept of **Political Narrative** is and to organize the pipeline for your research. Step by step, it helps you to define your topic, choose sources, identify characters, design prompts, and generate results, while highlighting the key questions you should answer at each stage.

Steps

Intro

1 • Topic

2 • Data

3 • Characters

4 • Prompts

5 • Coding

Step 3 — Identify relevant characters

Map the topic into relevant human and instrument characters.

Guide: character selection

Once your data is collected, the next step is to define the characters relevant to your topic. Characters can be human (e.g., individuals, corporations, parties, movements, states) or instrumental (e.g., policies, laws, technologies). Importantly, the term **character** refers to any entity playing a role in the narrative, it does not have to be a human or an individual. It can also include abstract or collective entities like “science,” a policy area, or a class of technology. Think of *character* as a functional category, not a synonym for *actor*.

At this stage, define a **small, distinctive set of characters** that directly reflects your research question. Favor **clear, non-overlapping definitions** that an LLM can reliably recognize in text. A good rule of thumb: could an outsider distinguish your characters based solely on their descriptions? Finally, **record a short description for each character now**—you’ll use these later when drafting prompts in Step 4.

How to approach

- As said above, each **character** should be seen as a category and not be mistaken with *actors*. The first step in defining characters is to read through your data. Before the LLM can encode narratives at scale, you need to make informed, grounded decisions based on what a human reader would plausibly identify.
- Identify relevant characters for the topic. You can base your selection on prior literature, your own research focus, or take a more data-driven approach. Characters can be identified manually, by reading your texts and noting recurring entities, or with the help of automated tools, though these are optional.
- Traditional tools such as word clouds or topic models can help. You may also use entity recognition with the more advanced [RELATIO package](#). However, all these methods eventually require dimensionality reduction: from the full set of entities they surface, you must decide how to group and narrow them down to a manageable, meaningful set of characters. Unlike topic modeling or RELATIO, in our framework, this aggregation step happens before the large-scale annotation: it is a necessary act of simplification that defines what the LLM will look for.
- Entity recognition is very useful for thinking about the necessary aggregation, as it can tell you which entities appear how often. This provides a good basis for thinking which entities should be aggregated into one character. For example, in real text the character **US president Trump** might appear as *Trump, Donald J. Trump, POTUS, US president*, etc. , but conceptually clearly represents just one character.
- A different aggregation task is to cluster several narrower characters into a broader one. For instance, you could have three separate characters *solar technology, wind technology, and geothermic technology* if you are interested in understanding specific narratives about these specific technologies. Or you can decide to aggregate all up into one character **green technology**, if your goal is to capture the narrative roles of green technology as a combined category. Similar examples would be looking at *primary education, secondary education, and tertiary education* separately, or having one character **education**. Hence one can think of a character as an aggregation of different words for the same clearly defined character, or of a larger character as a cluster aggregating several more specific characters.

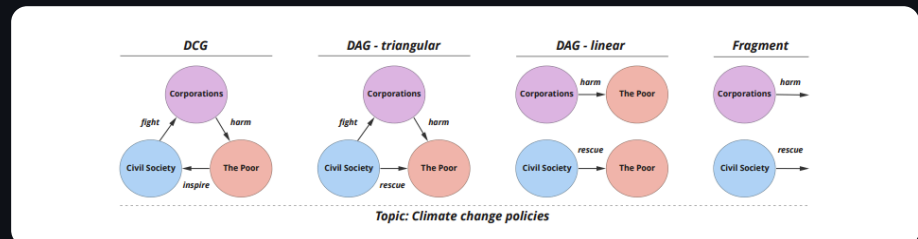
- For a precise prediction, characters should be internally as homogenous as possible, and as heterogenous and clearly differentiated as possible from other characters.
- Balance scope with feasibility: too many characters can raise computational costs and prediction complexity; a focused set improves reliability and interpretability. However, aggregating too many entities into one character will make a prediction more noisy, unless the mapping into the character is always obvious and clear. It can make sense to start with a larger set of characters which could be aggregated later for regression analysis, but be aware it may come with monetary costs and with a potential loss in precision.
- Document your choices and motivations for later. Also for each character, it helps to have both a short positive list of the key entities or more specific characters that it includes, as well as a negative list clarifying what it explicitly does not include.
- Go back and read a sufficient number of text snippets manually to validate your choices.
- Key is to anchor the selection to your research question and analytical focus; choose characters that speak to your hypotheses and that you want to use in your descriptives or regressions later.
- The list of characters doesn't need to include the whole universe of characters relevant for the topic, it just needs to be consistent with the scope of your research.

Ask yourself

- ☐ Which characters recur most often in prior literature or theory on this topic?
- ☐ Which characters matter most for the topic at hand?
- ☐ Do your characters appear in at least one drama triangle role (hero, villain or victim)? Or even in several? It can be interesting to observe if they only appear as neutral, but often more interesting if they can take on more roles
- ☐ Do exploratory tools (topic models, word clouds, entity recognition, RELATIO outputs) highlight additional entities? If so, how to aggregate these entities into some broader character definitions?

- ☐ What is the scope of your analysis (national, regional, global)? Which characters fall outside your scope?
- ☐ Do the selected characters exhibit distinctions that are sufficiently clear for an LLM to recognize and differentiate them?

Example — Relevant characters for climate change political discourse 💡



Characters as Nodes.

Political Narrative Characters for Climate Change Policy:

Guided by the relevant literature, exploratory tools, and intensive domain reading, we pre-specify ten characters: five human characters (made of institutions and groups of individuals) and five instrument characters (policy tools and instruments).

Human Characters: DEVELOPING ECONOMIES | US DEMOCRATS | US REPUBLICANS | CORPORATIONS | US PEOPLE

Instrument Characters: EMISSION PRICING | REGULATIONS | FOSSIL INDUSTRY | GREEN TECH | NUCLEAR TECH

We carefully decided to have ten character to balance the complexity of the analysis with good practices to avoid overloading the LLM in the prediction process. These characters are easily recognizable by the LLM thanks to precise descriptions that include positive examples and partly negative distinctions to related concepts. (Step 4).

Examples of Characters:

In our application, we were not interested in individual politicians, but in the distinction in democrats and republicans. The means one of our characters was **US Democrats**. This is then a cluster of the many different humans and the organization, e.g. members of

parliament, executive members, governors, etc., but also the democratic party itself. It also includes many different ways of referring to these more specific characters. We do not need to specify all of these, a couple of examples are sufficient for modern LLMs. Another example is the character **Green Technologies**. We were not interested in specific technologies, so this contains *clean tech sector*, *bioenergy*, *solar energy*, and similar entities. We noted some examples of technologies to provide them to the LLM in the next stages.

Characters as Nodes in DAG:

Global greenhouse emissions are still on the rise, oil production is soaring and **energy companies** are making sky-high profits while **countless people** struggle to pay their bills. [...] A critical mass of people – especially **younger people** – are demanding change and will no longer tolerate the procrastination, denial and complacency that created this state of emergency.

In the text and in the Figure *Characters as Nodes* it is clear how the characters can be seen as nodes of DAGs. The different entities in the text are identified within broader characters: corporations (*energy companies*), the poor (*countless people*), and civil society (*younger people*). However, the diagram shows that assigning causal arrows between characters may often be ambiguous in real texts. By contrast, assigning roles is typically clearer and can be coded directly: in this example, corporations are cast as villain, the poor as victim, and civil society as hero.

Output — Output - What you should have before Step 4 ⚠

- A concise table listing each character and its short description, ready to be inserted into Step 4 prompts.
- A brief description of each character should be included in the prompts at Stage 4. It is good practice to annotate these descriptions while selecting the characters. You can revisit and refine these descriptions once you see the prompt in Step 4.



Guide progress

- ☐ Step 1 — Topic
- ☐ Step 2 — Data
- ☐ Step 3 — Characters
- ☐ Step 4 — Prompts
- ☐ Step 5 — Outputs

A step turns green only when all its checkboxes are marked.

Political Narratives Guide

Use this guide to understand what the concept of **Political Narrative** is and to organize the pipeline for your research. Step by step, it helps you to define your topic, choose sources, identify characters, design prompts, and generate results, while highlighting the key questions you should answer at each stage.

Steps

Intro

1 • Topic

2 • Data

3 • Characters

4 • Prompts

5 • Coding

Step 4 — Prepare the prompt(s)

After defining your characters in Step 3, you now need to instruct the model on how to detect them. Step 4 translates your conceptual framework into an operational prompt that the LLM can understand.

Guide: prepare the prompt(s)

This step is crucial for achieving **accurate classification**. To ensure that the LLM assigns the correct roles to each character, you must provide **clear and precise instructions** in the form of a **SYSTEM MESSAGE**, formatted as a JSON file. Your SYSTEM MESSAGE should specify three components: first, the task the LLM must perform on each text snippet; second, the list of characters to identify, each with a brief description and, if useful, clarifying examples; and third, the set of possible roles they can assume — **Hero**, **Villain**, or **Victim**.

Below, we provide key guidance for constructing the SYSTEM MESSAGE. A ready-to-use example prompt is available in the [GitHub repository](#) and can be easily adapted by following these instructions.

To further improve data quality, we also include an optional **relevance classification** prompt. This step helps assess how well the

selected text snippets relate to your topic. Keyword-based filtering (used in Step 2) can be noisy and often includes texts that are not truly relevant. Such noise reduces data quality, making both classification and later analysis more difficult. The relevance classification prompt helps filter these texts out, saving both cost and time. If you choose to apply it, first run the **relevance classification** (Stage 1), and then apply the **character-role classification** (Stage 2) only to those texts labeled as relevant.

Step-by-Step Instructions:

1. Access the [GitHub repository](#) and download the `system_message_stage1` file for relevance classification and the `system_message_stage2` file for character-role classification from the `codes and prompts/` folder.
2. For the relevance classification (`system_message_stage1`), adjust the instructions in the SYSTEM MESSAGE to reflect your specific topic. No other changes are required for this prompt.
3. For the character-role classification (`system_message_stage2`), modify the instructions, update the list of characters, and provide their descriptions.

How to modify the two system messages? In the prompts, you will find the following text: *You are an average US citizen. The user will provide the content of a tweet posted from the US between 2010 and 2021. Your task is to analyze it within the context of US political discourse, particularly in relation to climate change and related policies.*

This example comes from our paper, where we focused on assessments by US citizens. Narratives are always context-dependent, and LLMs can capture this to some extent. We strongly recommend adapting this part to your target population. For instance, if your study focuses on India, you might write *You are an average Indian citizen*. If you analyze five European countries, you could either run separate prompts for each (to capture cultural nuances) or a single prompt reflecting an *average European perspective*. Ultimately, this choice depends on your research goals, but remember that context matters. Modify these few instructions to match your specific task. If you are working on character-role

classification, continue with the next steps; if you are only running relevance classification, you can proceed directly to Step 5.

4. In `system_message_stage2`, update the names and descriptions of your characters. In our example, we included ten characters, labeled with keys from `a` to `j`. Each character has a short description and an example. Replace these with your own topic-specific characters and descriptions.
5. After each description, specify the key where the result will be stored. The alphabetical keys should be lowercase, starting from `a` and continuing sequentially (e.g., `a-f` for six characters or `a-h` for eight). For instance, the classification output for the first character will be stored in column `a`, for the second in column `b`, and so on. These keys simply define how the LLM stores results in the output and can be modified without any knowledge of JSON syntax.

How to approach

- Define a **JSON schema** (keys, allowed labels, brief explanation of the task).
- Co-design prompts with the same model you will use for prediction (e.g., GPT-4o-mini via OpenAI) to align capabilities and outputs.
- This package supports two tasks: (a) topic relevance classification and (b) character detection + role assignment. Choose which task perform, and consider applying prompt 1 to increase the quality of your data.

Ask yourself

- ☐ Is the task singular and clear (avoid mixing multiple tasks in one prompt)? Use the correct prompt for the task that you need to perform.
- ☐ Is the chosen input unit appropriate for context vs. speed constraints? Remember, shorter text snippets are handled better by the OpenAI API.
- ☐ Is the schema unambiguous, machine-readable, and easy to parse? Are the keys correctly specified and the roles defined properly?

- ☐ Do I provide a description of the selected characters?
- ☐ If you copy-paste one test snippet into the model with this prompt (Chat gpt), do I get valid JSON with all keys? (If not, fix BEFORE Step 5.).

Example — Prompt design for the political narratives of climate change 💡

(a) Relevance classifier (Stage 1)

Decide if a three-sentence tweet/newspaper/other text source excerpt is relevant to **US climate change discourse**.

- 0 = irrelevant (no meaningful climate discussion)
- 1 = assert (asserts existence of climate change)
- 2 = deny (denies or mocks climate change)
- 3 = relevant (substantive discussion of climate change policy)

We used this script to filter the downloaded tweets and to keep only those that are actually relevant for the climate change policy discourse. This step can be seen as complementary to the topic selection step, meaning that this increases data quality and relevance to the topic. This is useful to avoid useless classification of irrelevant tweets and hence save money and time.

(b) Character detection & role assignment (Stage 2)

Identify pre-specified characters and assign contextual roles:

- Villain (1) | Hero (2) | Victim (3) | No role (4)

Characters: Developing Economies, US Democrats, US Republicans, Corporations, US People, Emission Pricing Tools, Regulation Policies, Fossil Fuels, Green Technologies, Nuclear Energy. Output JSON contains keys a–j (one per character).

In the prompt, for each character we give a brief description and some positive examples to look for in the text.

Output — What you should have before Step 5 ⚠️

- One or two prompts, adapted to your topic, your location/country/culture and your characters. The prompt `system_message_stage1` for the relevance classification contains

the system message with the instructions and the categories in which a text can be classified (coded from 0 to 3). The `system_message_stage2` contains the system message with the instructions, the list of characters that you selected in Step 3, and the description of each character.

- You now have both prompts ready. In the next step, you'll run the provided code to generate your annotated dataset.

Examples of JSON prompts

Provide machine-readable JSON specs for Stage 1 (relevance classification) and Stage 2 (character/role assignment). You can find the original prompts for *Gehring and Grigoletto (2025)* in the GitHub repository.

✿ (a) Stage 1 — Relevance prompt JSON

```
{
  "SYSTEM_MESSAGE": "You are an average US citizen. The user will provide the content of a three-sentence newspaper article excerpt published by a US newspaper between 2010 and 2021. Your task is to analyze the excerpt within the context of US political discourse, particularly in relation to climate change. Respond in JSON format.

  1. Relevance Check: Analyze the excerpt in the context of US climate change discussion and determine its relevance. Provide one of the following values:
    - 0 (irrelevant): If the excerpt does not discuss climate change in a meaningful way. For example, if it only includes a hook tag like
```

✿ (b) Stage 2 — Character & role prompt JSON

```
{  
  "SYSTEM_MESSAGE": "You are an average US citizen. The user will  
  provide a three-sentence US newspaper excerpt (2010–2021).  
  Analyze it in the context of US political discourse on climate change  
  and respond in JSON format.
```

Personal notes for Step 4 (optional)

1. Character Analysis: For each mentioned character, assign a role:

- 3 = Victim: suffers unfairly, is attacked, or endures consequences of climate change or others' actions.
- 4 = No role: mentioned but not clearly cast as Villain/Hero/Victim, or context is neutral/ambiguous.

2. Characters to evaluate (keys a–j):

- a: Developing Economies — emerging and poorer nations (incl. BRICS). Mentions of their governments, representatives, or citizens in the context of climate negotiations or responsibilities.

- b: US Democrats — politicians or public figures tied to the

Guide progress

- ☐ Step 1 — Topic
- ☐ Step 2 — Data
- ☐ Step 3 — Characters
- ☐ Step 4 — Prompts
- ☐ Step 5 — Outputs

A step turns green only when all its checkboxes are marked.

Political Narratives Guide

Use this guide to understand what the concept of **Political Narrative** is and to organize the pipeline for your research. Step by step, it helps you to define your topic, choose sources, identify characters, design prompts, and generate results, while highlighting the key questions you should answer at each stage.

Steps

Intro

1 • Topic

2 • Data

3 • Characters

4 • Prompts

5 • Coding

Step 5 — Obtain predictions and assemble outputs

Run prediction, parse JSON, and build tidy outputs (stage flags, presence, role dummies).

Guide - GPT prediction with provided code

This is the final step of the pipeline. You will now run the prediction code that applies your prompts to your dataset. This page of the guide will help you adapting the scripts that we provide in the Github repository to your objective. In practice, this step is where you will apply the model instructions finalized in Step 4 to your text data, producing the full narrative prediction.\n

Step-by-Step instructions:

1. (For beginner Python users) First, we suggest you to download Anaconda and Anaconda Prompt at the [link](#). Anaconda already includes Python, so you don't need to install Python separately.
2. Download the files in `code and prompts/` and the `environment.yml` from the same folder in Github. Then, build the following folder structure locally:

```

<main>/
├── code/
│   ├── annotation_openai_stage1.py
│   ├── annotation_openai_stage2.py
│   └── prompts/
│       ├── system_message_stage1.json      # <-- you
│       └── system_message_stage2.json      # <-- you
├── data/
│   └── output/
│       ├── your_data_stage1.csv            # <-- you
│       └── your_data_stage2.csv            # <-- you
├── output/
│   ├── data/
│   │   ├── openai_output/
│   │   └── openai_final/
├── logs/
└── environment.yml                        # <-- you

```

According to this structure, you need to fill the `code/prompts/` folder with the system message of the stage, and the `data/output/` folder with the `.csv` dataset. Put the `environment.yml` in the main project folder.

IMPORTANT: the input `.csv` must contain a column called `id` and a column called `text`.

3. Open the app **Anaconda Prompt** and move into the main project folder. To open Anaconda Prompt, you need to search the app in your machine and click on it. Once opened, write in the black command window the following:

```
cd C:\Users\YourName\Documents\political_narrative_p
```

4. Then, create the environment where the code will run by using the following command:

```
conda env create -f environment.yml
```

5. Activate the environment. Notice, in this environment we also specify the version of Python that is needed for the script to run.

```
conda activate political_narrative
```

6. Install the correct version of `openai`

```
pip install "openai==2.4.0"
```

7. Set the `OPENAI_API_KEY` as an environmental variable in your current environment. Run the following commands once at a time:

```
conda env config vars set OPENAI_API_KEY="sk-your-key"
conda deactivate
conda activate political_narrative
```

This step is crucial for the success of the annotation process. This code requires an individual OpenAI API key, that the user can retrieve in his [OpenAI personal page](#). This key is personal and directly linked to the user's wallet, so it's important to keep it personal and hidden in the machine and not in the script.

8. Open Anaconda Prompt and activate Spyder or your preferred Python IDE (here an example using Spyder) by running the commands:

```
conda install spyder python=3.13.5
spyder
```

9. Open the script of interest directly in spyder using the top left command bar. Here you can choose one of the two Python scripts depending on the task that you need to perform:

A) The **stage 1 script** `annotation_openai_stage1` allows to classify a text based on its relevance to the topic selected. This code returns an additional column to the input dataset that takes values from 0 to 3 (0 - irrelevant, 1 - assert, 2 - deny, 3 - relevant). This script is not strictly necessary for the character-roles annotation, but it can be useful to assess the relevance of a specific text to the topic at hand. For example, in *Gehring and Grigoletto (2025)* we use this script to

filter the tweets and keep only those relevant to the topic of climate change policy.

B) The **stage 2 script** `annotation_openai_stage2` is the core of the Political Narrative Package, and it allows to retrieve the character-role classification. The code returns a dataset with a column for each specified character taking values from 0 to 4, where 0 is no-mention of the character, 1 is Villain role, 2 is Hero role, 3 is Victim role, and 4 for appearance of the character in none of these roles (Neutral).

10. Regarding the prompt, you can adapt the prompt in the folder `code/prompts/` with your own instructions. You can do this by accessing the "SYSTEM MESSAGE" in the prompt. If you need more details about the prompt design, please go back to Step 4 of this guide.
11. Modify the input dataset: you need to provide in the `/data/output` folder your dataset named `your_data_stage2` (or `your_data_stage1`). This dataset must contain a column called `id` with the unique identifiers and a column called `text` containing the text for the classification. The `.csv` file must be UTF-8 encoded. If you see errors such `unicodeDecodeError` try saving again your file in UTF-8, or the script will automatically fall back to latin-1 encoding.
12. Once prepared the folder structure, the prompt, and the dataset, you can make minimal changes to the Python script. First, changing the directory

```
main = r"D:\your directory"
```

If you are running the `annotation_openai_stage1` code, no other changes are needed and you can directly run the script.

13. Second, change the JSON structure based on the number of characters: depending on your character selection, you might need to modify the structure of the `.JSON` file that is created by OpenAI. Here, you just need to adapt the number of keys to your number of characters. You can access it by changing the properties in the following part of the script:

```

JSON_SCHEMA = {
    "name": "ArticleClassification",
    "schema": {
        "type": "object",
        "properties": {
            "items": {
                "type": "array",
                "items": {
                    "type": "object",
                    "properties": {
                        "id": {"type": ["integer", "string"], "required": True},
                        "a": {"type": "integer", "required": True},
                        "b": {"type": "integer", "required": True},
                        "c": {"type": "integer", "required": True},
                        "d": {"type": "integer", "required": True},
                        "e": {"type": "integer", "required": True},
                        "f": {"type": "integer", "required": True},
                        "g": {"type": "integer", "required": True},
                        "h": {"type": "integer", "required": True},
                        "i": {"type": "integer", "required": True},
                        "j": {"type": "integer", "required": True},
                    },
                    "required": ["id", "a", "b", "c", "d", "e", "f", "g", "h", "i", "j"],
                    "additionalProperties": False
                }
            }
        },
        "required": ["items"],
        "additionalProperties": False
    },
    "strict": True
}

```

14. Then, you need to change the user content to adapt it to your number of characters, as done before:

```

def build_user_content(article_group):

    user_content = (
        "Classify the following opinion(s) strictly per the schema below.\n"
        "Respond with only a JSON object of the form: {\"items\": [...]}\n"
        "where \"items\" is an array of objects (one per opinion).\n"
        "Each object must include keys: id, a, b, c, d, e, f, g, h, i, j.\n"
        "No extra text.\n\n"
    )

```

```
)
for article in article_group:
    user_content += f"ID: {article['id']}\n"
    user_content += f"Text: {article['text']}\n"
return user_content
```

15. Lastly, you need to adapt the `flatten_results()` function by changing this few lines of code:

```
"request_id": result["request_id"],
"id": entry_id,
"text": result.get("article_text", None),
"a": entry.get("a", 0),
"b": entry.get("b", 0),
"c": entry.get("c", 0),
"d": entry.get("d", 0),
"e": entry.get("e", 0),
"f": entry.get("f", 0),
"g": entry.get("g", 0),
"h": entry.get("h", 0),
"i": entry.get("i", 0),
"j": entry.get("j", 0)
```

16. It's finally time to run the code. You can do this either by highlighting the whole script and clicking `F9` on the keyboard, or by clicking on the green triangle at the top of the Spyder interface. Your results are saved in `.csv` format as `flattened_results_all.csv`. These are stored in the local folder `main/output/data/openai_final/run20251027_133424/`, where the last part (`run20251027_133424`) takes his name from the date and time when the script has been compiled (`date_time`). This `.csv` file contains the results of the Openai classification.

Settings of the Model

Depending on the parameters, the annotation will return different results. In this package, you are allowed to change the **temperature** of the model by accessing the temperature field in the

`send_request()` function:

```
def send_request(request_id, article_group, system_m
```

```

user_content = build_user_content(article_group)

body = {
    "model": "gpt-4o",
    "messages": [
        {"role": "system", "content": system_mes
        {"role": "user", "content": user_content
    ],
    "max_tokens": 300,
    "temperature": 0.5, # <----- ACCESS IT FROM
    "response_format": {
        "type": "json_schema",
        "json_schema": JSON_SCHEMA
    }
}

```

Temperature is a parameter that controls the “creativity” or randomness of the text generated by the GPT model. A higher temperature (e.g., 0.7) results in more diverse and creative output, while a lower temperature (e.g., 0.2) makes the output more deterministic and focused.

In practice, temperature affects the probability distribution over the possible tokens at each step of the generation process. A temperature of 0 would make the model completely deterministic, always choosing the most likely token.

Lastly, the user can also decide which GPT model to use. This code allows for **gpt-4o**, **gpt-4o-mini**, **gpt-4.1**, and **gpt-4.1-mini**.

Ask yourself

- ☐ Is your dataset ready for the prediction (id, text, missing values, etc)?
- ☐ Does the dataset contain a column called "id" and a column called "text"?
- ☐ Are your prompts correctly specified and specific enough?
- ☐ Is your folder structure organised as expected? To check this point we suggest you to visit the github repository and the instructions listed there.

Example — Prediction output and relevant texts 💡

- ☐ Are you in the correct environment (Political_Narratives)?

In the OpenAI API key set at the environment?

corresponds to a character (a–X) and contains numeric role codes: 1 = Villain, 2 = Hero, 3 = Victim, 4 = No role:

- Character presence (which characters appear)
- Role indicators (whether a character is assigned a role)

A text is defined as a **political narrative** if at least one characters has an assignment of a drama triangle role; if all are neutral we do not categorize it as a narrative.

Output — What you should have at the end 🌟

- A dataset with your text snippets as the rows, with the columns indicating your characters, and the cell values indicating the roles. X depends on the number of characters selected: with six characters, X=f, and the columns run from a-f. At this point your dataset is fully annotated and ready for analysis or visualization:



id	text	a	b	c	d	e	f	...	X
1_1	If you listen...	1	4	0	0	0	0		0
1_2	But is it po...	0	4	0	2	2	0		0
1_3	In fact, on...	3	1	0	0	0	0		0

Personal notes for Step 5 (optional)